

Modeling Computation

R Pruim

2019-04-08

Contents

Preface	2
1 Alphabets, Languages, and Grammars	1
1.1 Terminology	1
1.2 (Phrase-Structure) Grammars	1
2 Types of Grammars	3
2.1 Backus-Naur Form (BNF) for Context Free Grammars	3
3 Finite State Automata	5
3.1 Graph Representation	5
3.2 Extended transition function	5
3.3 Language Recognition	6
3.4 Equivalent Automata and Grammars	7
3.5 Nondeterministic Automata	7

Preface

These exercises were assembled to accompany a Discrete Mathematics course at Calvin College.

1 Alphabets, Languages, and Grammars

1.1 Terminology

- An **alphabet** (or **vocabulary**) is just a finite, non-empty set. It's elements are called **letters** or **symbols**.
- A **word** (or **string**) is a finite sequence of symbols.
- A word consisting of no symbols is called the **empty word** and denoted λ . (Think "".)
- The set of all words using exactly n symbols from V (repetition allowed) is denoted V^n .
- The set of all words using symbols in alphabet V is denoted V^* . (So $V^* = V^0 \cup V^1 \cup V^2 \dots$.)
- A **language** over V is a subset of V^* .

Exercises

1. Let $V = \{0, 1\}$. What is V^0 ? What is V^2 ? What is V^* ?
2. Let $V = \{2\}$. What is V^0 ? What is V^2 ? What is V^* ?
3. Can $\emptyset$ (empty set) be a letter? an alphabet? a word? a language?

1.2 (Phrase-Structure) Grammars

A phrase-structure grammar consists of

- **alphabet:** V
- **start symbol:** $S \in V$
- **terminal symbols:** $T \subset V$
 - $N = V - T$ is the set of **nonterminal symbols**
- **production rules:** $P \subseteq (V^* - N^*) \times V^*$
 - usually write elements of P as $u \rightarrow v$ rather than (u, v) .
 - $V^* - N^*$ says that u must contain at least one nonterminal.
 - v can be any combination of terminals and nonterminals (including the empty string).

Examples

Note: in each of these examples, capital letters are used for nonterminals and lower case letters or digits for terminals. That makes it easy to remember, but it is not a requirement of the definition of a grammar.

Grammar 1 (G_1): alphabet: $\{a, b, A, B, S\}$, terminals: $\{a, b\}$, start symbol: S , production rules:

- $S \rightarrow ABa$
- $A \rightarrow BB$
- $B \rightarrow ab$
- $AB \rightarrow b$

Grammar 2 (G_2): alphabet: $\{S, A, a, b\}$, terminals: $\{a, b\}$, start symbol: S , production rules:

- $S \rightarrow aA$
- $S \rightarrow b$
- $A \rightarrow aa$

Grammar 3 (G_3): alphabet: $\{S, 0, 1\}$, terminals: $\{0, 1\}$, start symbol: S , production rules:

- $S \rightarrow 11S$
- $S \rightarrow 0$

Grammar 4 (G_4): alphabet: $\{S, A, B, C, a, b, c\}$, terminals: $\{a, b, c\}$, start symbol: S , production rules:

- $S \rightarrow AB$
- $A \rightarrow Ca$
- $B \rightarrow Ba$
- $B \rightarrow Cb$
- $B \rightarrow b$
- $C \rightarrow cb$
- $C \rightarrow b$

1.2.1 Derivations and Languages

The rules of a grammar are used to derive strings of terminals (elements of T^*) as follows.

- If $w_0 \rightarrow w_1$ is a rule, then $lw_0r \Rightarrow lw_1r$ for any strings l and r .
- $a \xRightarrow{*} b$ is defined recursively. $a \xRightarrow{*} b$ if either
 - $a \Rightarrow b$, or
 - $\exists c (a \Rightarrow c \wedge c \xRightarrow{*} b)$

Basically the production rules are “rewrite rules” that allow us to replace the left side of the rule with the right side. A derivation is a sequence of rewrites.

The language of a grammar G is denoted $L(G)$ and contains all strings of terminals that can be derived from the start symbol:

$$L(G) = \{w \in T^* \mid S \xRightarrow{*} w\}$$

1.2.1.1 Exercises

4. Show that using Grammar 1, $ABa \xRightarrow{*} abababa$.
5. Is $abababa \in L(G_1)$, the language generated by Grammar 1?
6. What is $L(G_2)$?
7. What is $L(G_3)$?
8. Generate several words using G_4 .
9. Create a grammar G_5 such that $L(G_5) = \{0^n 1^n \mid n = 0, 1, 2, \dots\}$
10. Create a grammar G_6 such that $L(G_6) = \{0^n 1^n \mid n \in \mathbb{Z}^+\}$
11. Create a grammar G_7 such that $L(G_7) = \{0^m 1^n \mid m, n \in \mathbb{N}\}$

2 Types of Grammars

Type 0: no restrictions

Type 1 (context sensitive): only two types of rules allowed

- $lAr \rightarrow lwr$ where
 - $l, r \in V^*$
 - $A \in N$
 - $w \in V^+$ [That is, $w \neq \lambda$.]
 - shorthand: `[left] [nonterminal] [right] → [left] [non-empty] [right]`
- $S \rightarrow \lambda$
 - shorthand: `[start] → [empty string]`
 - If the rule is used, the the start symbol may not appear on the right side of any rule, so other rules become `[left] [non-empty without start symbol] [right]`.

Type 2 (context free): only one type of rule allowed

- $A \rightarrow w$ where
 - A is a single nonterminal symbol
 - $w \in V^*$ (so no restriction here)
- shorthand: `[nonterminal] → [any string]`

Type 3 (regular): Three types of rules allowed

- `[nonterminal] → [terminal] [non terminal]` (Example: $A \rightarrow bC$)
- `[nonterminal] → [terminal]` (Example: $A \rightarrow b$)
- `[start] → [empty string]` (Example: $S \rightarrow \lambda$)

A regular/context free/context sensitive language is a language that can be generated by a regular/context free/context sensitive grammar.

1. For each grammar we have seen so far, determine whether it is regular, context free, context sensitive. (Some grammars will be more than one. All grammars are type 0.)
2. True or false: Every grammar of type n is a grammar of type $n - 1$.

2.1 Backus-Naur Form (BNF) for Context Free Grammars

Shorthand notation for type 2 grammars.

- nonterminals denoted with $\langle \rangle$.
- $\rightarrow$ written as `::=`
- All rules with same nonterminal on left written together with the list of possible right sides separated by `|`.

Example: If we have production rules $A \rightarrow Aa$, $A \rightarrow a$, and $A \rightarrow AB$, we will write

`<A> ::= <A>a | a | <A><B>`

BNF for ALGOL 60 identifier

```
<identifier> ::= <letter> | <identifier><letter> | <identifier><digit>
<letter> ::= a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|r|s|t|u|v|w|x|y|z
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

BNF for signed integer

```
<signed integer> ::= <sign><integer>
<sign> ::= + | -
<integer> ::= <digit> | <digit><integer>
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

BNF specification for Java

You can find BNF for various programming languages online. For example: <https://users-cs.au.dk/amoeller/RegAut/JavaBNF.html> has a BNF specification for Java.

Exercises

3. Why does BNF notation only work for context free grammars?
4. For each context free grammar we have seen, give its BNF representation.

3 Finite State Automata

A **finite state automaton** consists of the following:

- a finite set of **states**: S
- a finite **input alphabet**: I
- a **start state**: $s_0 \in S$.
- a set of **final states**: $F \subseteq S$
- a **transition function**: $f : S \times I \rightarrow S$

Example (M_0) states: $\{A, B, C, D\}$; input alphabet: $\{0, 1\}$; start state: A ; final states: $\{A, D\}$; transition function described in table below

M_0								
state	A	A	B	B	C	C	D	D
letter	0	1	0	1	0	1	0	1
f	A	B	A	C	A	A	C	B

3.1 Graph Representation

1. It is often easier to visualize what is going on if we represent an automaton with a graph. Create a labeled, directed graph with the following properties:
 - There is a vertex for each state, labeled with the state. (Draw this as a circle with the state inside the circle.)
 - There is an edge from state s to state t labeled with letter x if and only if $f(s, x) = t$
 - Final states are circled a second time. (We could use shape or color or something else to make it clear which states are final states, but double circling is easy.)
 - An extra arrow (not coming from any state) points to the start state. This isn't really an edge in the graph, just an extra bit of labeling.

3.2 Extended transition function

We can extend the transition function to $f^* : S \times I^* \rightarrow S$ with the following recursive definition:

- $f^*(s, \lambda) = s$ for any state s
 - $f^*(s, xa) = f(f^*(s, x)a)$ for any $x \in I^*$ and $a \in I$
2. Use this definition to determine the following.
 - a. $f^*(B, \lambda)$
 - b. $f^*(B, 0)$
 - c. $f^*(B, 010)$
 - d. $f^*(A, 101)$
 - e. $f^*(A, 1011)$

3.3 Language Recognition

For any automaton M with transition function f , start state s and final states F , the language recognized by M (written $L(M)$) is defined as follows:

$$x \in L(M) \iff f^*(s, x) \in F$$

3. For each string x below, compute $f^*(A, x)$. Which of these strings are in $L(M_0)$?
(We will say that such strings are **accepted by** M_0 .)

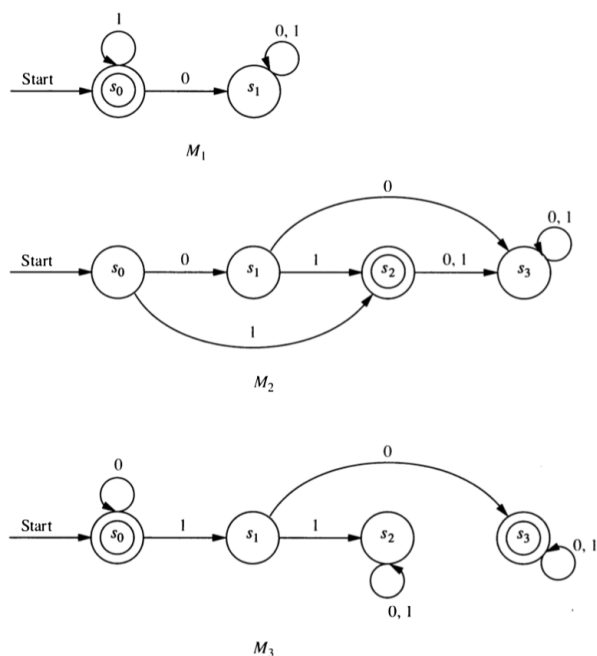
- a. λ
- b. 0
- c. 1
- d. 010
- e. 1011

Here are three automata:

4. For each of the machines M_1, M_2, M_3 , compute

- a. $f^*(s_0, 10)$
- b. $f^*(s_0, 1011)$
- c. $f^*(s_1, 1011)$

5. For each of the machines M_1, M_2, M_3 , determine the language recognized.



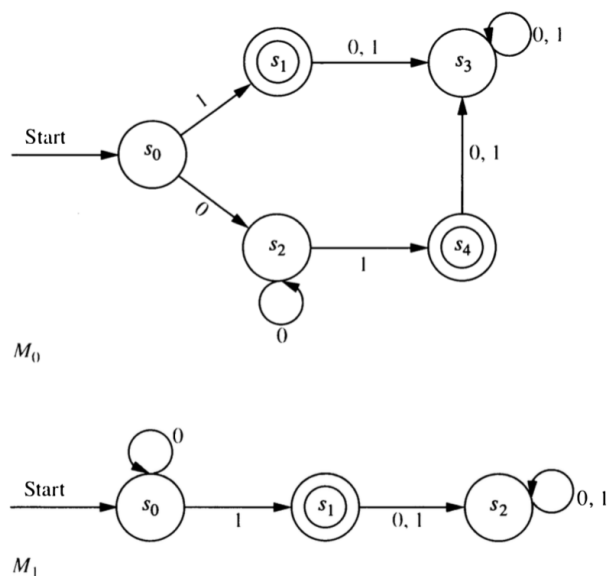
6. Create automata that recognize each of the following languages.

- a. The set of bitstrings that begin with two 0's
- b. The set of bitstrings that contain exactly two 0's
- c. The set of bitstrings that contain at least two 0's
- d. The set of bitstrings that contain two consecutive 0's (anywhere in the string)
- e. The set of bitstrings that do not contain two consecutive 0's anywhere
- f. The set of bitstrings that end with two 0's

3.4 Equivalent Automata and Grammars

We will say that two automata M and N are equivalent if $L(M) = L(N)$, that is, if they recognize the same language. Similarly, two grammars are equivalent if they generate the same language.

7. What must you do to show that two automata are **not** equivalent?
8. What must you do to show that two automata **are** equivalent.
9. Are the two automata represented below equivalent? Explain.



3.5 Nondeterministic Automata

The finite state automata we have seen so far are often called **deterministic finite-state automata** or DFAs. There is a generalization called a **non-deterministic finite-state automaton** or NFA. The only difference is how the transition function is specified. For an NFA, the transition function has the form

$$f : S \times I^* \rightarrow P(S)$$

where S is the set of states, I is the input alphabet, and $P(S)$ is the powerset of S . (The powerset of S is the set of all subsets of S . So the transition function for an NFA returns a *set* of states.)

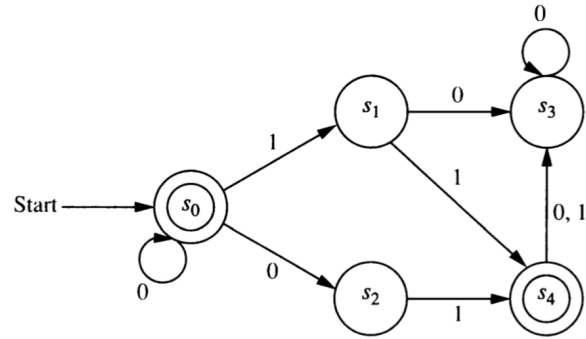
Here is a tabular description of the transition function for an NFA N_0 with start state A and final states C and D.

N_0								
state	A	A	B	B	C	C	D	D
letter	0	1	0	1	0	1	0	1
f	$\{A, B\}$	$\{D\}$	$\{A\}$	$\{B, D\}$	$\{\}$	$\{A, C\}$	$\{A, B, C\}$	$\{B\}$

10. Draw the graph representation. How will the graph of an NFA differ from the graph for a DFA?
11. How do we define the language recognized by an NFA? [Check your answer before proceeding.]
12. Which of these strings are accepted by N_0 ?

- a. 01
- b. 011
- c. 00
- d. 0000
- e. λ
- f. 010101

Here is the graph of an NFA N_1 .



- 13. Make a table showing the transition function for N_1 .
- 14. Which of the following strings are accepted by N_1 ?
 - a. λ
 - b. 00
 - c. 01
 - d. 010
 - e. 011
 - f. 001
- 15. What is $L(N_1)$?
- 16. Create a DFA that recognizes $L(N_1)$.
- 17. Given an NFA N , is it always possible to create a DFA M that recognizes the same language? If so, explain how. If not, provide an example N and explain why $L(N)$ cannot be recognized by a DFA.